



UNIVERSITÀ
DEGLI STUDI
FIRENZE

Da un secolo, oltre.



HR EXCELLENCE IN RESEARCH

03.4: Distributed Machine Learning

Network Applications

LM in Ingegneria Informatica

Daniele Tarchi

Dipartimento Ingegneria dell'Informazione

Advanced Network Application Paradigms

This 26-hour module is a deep dive into the next generation of network technologies, structured into four key topics that explore how networks are being transformed to support modern, demanding applications.

Part 1: Mobile Edge Computing (MEC) (7 hours)

- This topic explores **Mobile Edge Computing (MEC)**, a distributed paradigm that brings cloud capabilities to the network's edge. We'll see how MEC drastically reduces latency by processing data locally, enabling real-time applications like **AR**, **VR**, and **video analytics** that are impossible with traditional centralized clouds.

Part 2: Network Slicing (7 hours)

- This topic explores **Network Slicing**, a key 5G technology that partitions a single physical network into multiple virtual "slices." Each slice can be customized for specific performance requirements like bandwidth and latency, serving applications from **IoT devices** to **autonomous vehicles**.

Part 3: Multimedia Networking (7 hours)

- This topic is dedicated to the challenges of delivering high-quality multimedia content. We'll cover protocols and techniques for ensuring quality of service (QoS) and managing jitter and latency. Key topics include **adaptive bitrate streaming** and the role of **CDNs**.

Part 4: Distributed Machine Learning (DML) (6 hours)

- This topic explores **Distributed Machine Learning (DML)**, where ML models are trained and run across a network of devices. It enables local computation, reducing reliance on a central server. We'll discuss **federated learning**, where a model is trained on devices without sharing raw data, and its benefits for **personalized assistants** and **IoT analytics**.

Foundations of DML and Federated Learning



Why Machine Learning in Networks?

Let's summarize the necessity of ML in modern networking where traditional deterministic algorithms reach their scalability and complexity limits.

1. The Complexity Gap

- Traditional networking relies on "if-then-else" logic and hard-coded heuristics. As we move toward 6G, the number of variables (beamforming, ultra-dense cells, slice isolation) becomes too high for manual modeling. ML is used for tasks that are **too high-dimensional or dynamic** for standard algorithms.

2. Prediction (Anticipating Future States)

- **Traffic Forecasting:** Predicting temporal and spatial load to proactively scale resources.
- **QoE Prediction:** Estimating the subjective Quality of Experience based on objective physical layer metrics (KPI-to-KQI mapping).
- **Mobility Prediction:** Using historical paths to predict the next cell handover, reducing "ping-pong" effects and handover failure.

Why Machine Learning in Networks?

3. Classification (Pattern Recognition)

- **Anomaly Detection:** Distinguishing between normal network behavior and hardware/software faults.
- **Security Threat Identification:** Detecting zero-day attacks, DDoS signatures, and encrypted malicious traffic.
- **Traffic Classification:** Identifying application types (e.g., distinguishing Zoom vs. YouTube vs. Gaming) to apply specific QoS policies.

4. Control & Optimization (The "Closed-Loop")

- **Real-time Resource Allocation:** Dynamically assigning PRBs (Physical Resource Blocks) to network slices and managing MEC compute capacity.
- **ABR Algorithm Optimization:** Enhancing Adaptive Bit Rate logic for video streaming to minimize re-buffering in fluctuating channel conditions.
- **Energy Efficiency:** Intelligently powering down sub-systems (e.g., MIMO antennas) during low-traffic periods without violating SLAs.

ML transforms the network from a **Reactive** system into a **Proactive and Autonomous** infrastructure.

The Failure of Centralized ML



There are fundamental physical, temporal and legal barriers that make traditional centralized AI/ML architectures **unsustainable for 5G/6G and massive IoT ecosystems.**

1. The Data Gravity / Bandwidth Wall (The Physical Barrier)

- **The Concept:** As datasets grow into the Petabyte and Exabyte scale, they develop **"Data Gravity."** This means the cost and time required to move data toward compute resources exceed the benefits of the analysis.
- **The Bottleneck:** While 5G increases wireless capacity, the **backhaul and core network** remain expensive shared resources.
- **Infrastructure Impact:**
 - Base stations and IoT gateways generate a continuous telemetry stream that would saturate the uplink if sent in raw form.
 - **Strategic Failure:** Moving data to the cloud for "centralized learning" often results in the network becoming its own worst enemy—**consuming the bandwidth it is trying to optimize.**

The Failure of Centralized ML

2. The Latency Wall (Speed of Light & Real-Time AI)

- **The Concept:** Intelligence must be delivered within the application's "Action Window."
- **The Problem:** Traditional centralized ML requires a Round-Trip Time (RTT) from the Edge → Cloud → Edge.
 - Central Cloud RTT: **50ms – 150ms.**
 - Tactile Internet Requirement: **< 10ms.**
- **The Impact:**
 - For applications like **Autonomous Vehicle Collision Avoidance** or **Surgical Robotics**, the latency of the cloud is effectively "infinite"—the decision arrives after the event has occurred.
 - Intelligence must be "Instantaneous," requiring training and inference to happen at the Edge.

The Failure of Centralized ML

3. The Privacy Wall (The Legal & Ethical Barrier)

- **The Problem:** Edge data is inherently personal. It contains precise location history, web behavior, physiological data from wearables, and environmental audio/video.
- **The Risks:**
 - **Centralized Vulnerability:** Aggregating data into a "Data Lake" creates a high-value target for cyberattacks (the "Honey Pot" effect).
 - **Regulatory Friction:** Moving data across borders or even out of a user's device can trigger strict legal violations.
- **Key Regulations:**
 - **GDPR (EU):** Strict mandates on data minimization and user consent for data transfer.
 - **HIPAA (USA):** Absolute protection requirements for healthcare data generated by medical IoT.
 - **Data Sovereignty:** Many nations now prohibit the storage of citizen data on foreign cloud servers.
- **The Impact:**
 - In many jurisdictions, it is **legally prohibited** to move raw personal data across borders or even out of a user's device without explicit, granular consent.



The Failure of Centralized ML

The Strategic Shift

- The "Three Walls" mean that the traditional cloud model is not just inefficient—it is **physically and legally impossible** for the next generation of intelligent services.
- To overcome these walls, the industry is moving toward **Distributed Machine Learning (DML)**.
- The mantra has shifted from:
 - *Old Paradigm*: "Bring the data to the model."
 - *New Paradigm*: "**Bring the model to the data.**"

This transition represents the evolution from "Big Data" (volume-focused) to "Wide Data" (distributed-focused).

The Paradigm Shift: An Introduction to Distributed ML

Let's introduce Distributed Machine Learning (DML) and establish the fundamental principle that differentiates it from traditional centralized learning.

1. The Core Principle

- **"Bring the computation to the data, instead of bringing the data to the computation."**
- **Centralized ML:** Data is collected, transported, and stored in a central repository (Cloud/Data Center) for processing.
- **Distributed ML:** The learning algorithm is dispatched to the edge/nodes. Local data is processed locally, and only model parameters (not raw data) are synchronized.

The Paradigm Shift: An Introduction to Distributed ML

2. DML Taxonomy

- Distributed learning strategies are generally categorized based on what is being partitioned:
- **A. Data Parallelism**
 - **Concept:** The dataset is split into "shards" across multiple nodes.
 - **Operation:** Each node trains a local copy of the full model on its specific subset of data.
 - **Context:** Most common in high-performance cloud clusters to reduce training time.
- **B. Model Parallelism**
 - **Concept:** The model architecture itself is too large for a single machine's memory.
 - **Operation:** Different layers or components of the model are hosted on different nodes.
 - **Context:** Used for training massive LLMs or complex neural networks with billions of parameters.

Beyond Distributed Training: Three Edge-AI Paradigms

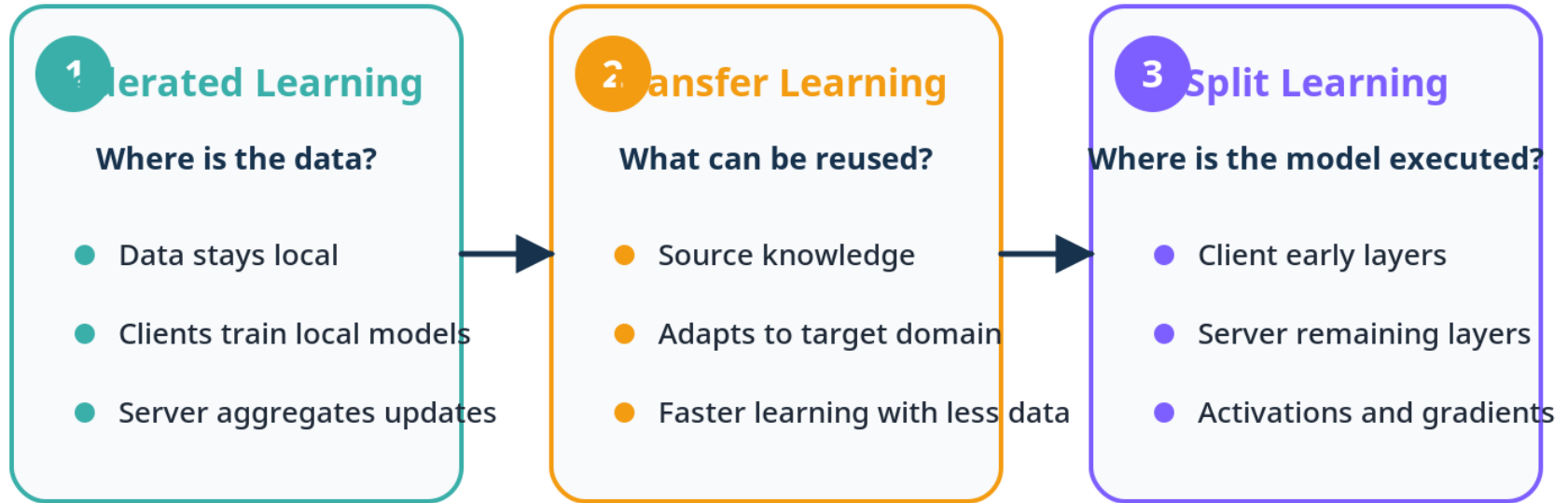
Distributed ML is not a single technique. It is a family of strategies for moving learning closer to data, devices, and network dynamics.

Distributed Machine Learning can be organized around three complementary questions:

- **1. Where is the data?**
Federated Learning answers this by keeping data local and exchanging model updates.
- **2. What previous knowledge can be reused?**
Transfer Learning answers this by adapting knowledge from a source domain/task to a new target domain/task.
- **3. Where is the neural network executed?**
Split Learning answers this by partitioning the model itself between client and server.

Key message: First we study Federated Learning as the main DML paradigm; then we enlarge the view to Transfer Learning and Split Learning

DML roadmap: from distributed data to reusable and split intelligence



Federated Learning (FL)



Core Concept: Moving the Model, Not the Data

1. The Medical Analogy (Privacy-First Intelligence)

Imagine a group of medical specialists who want to train an AI to detect a rare disease.

The Problem: They cannot share patient records with each other due to strict privacy laws (GDPR/HIPAA) and ethical concerns.

The FL Solution:

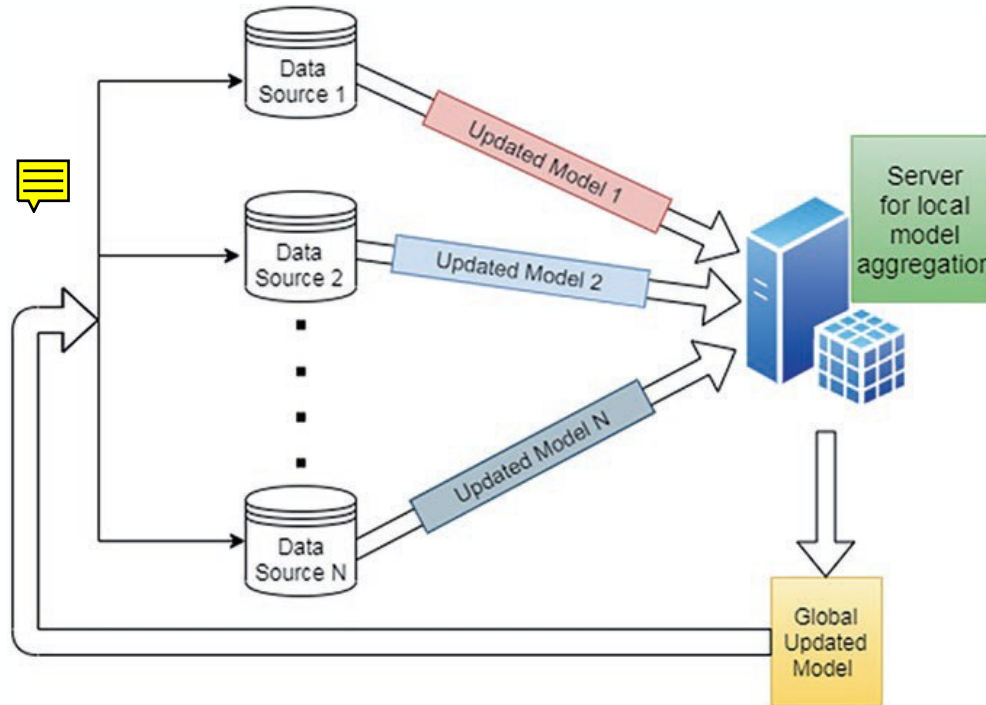
1. A central server sends a **base model** to each hospital.
2. Each doctor "trains" the model locally on their own private patient data.
3. Instead of sending patient files, the doctors send back only the **mathematical updates** (insights) to the server.
4. The server aggregates these small updates to create a **superior global model**.

Federated Learning (FL)

2. The Operational Workflow

Phase	Action	What Moves?
1. Distribution	The central server broadcasts the current model to all nodes (hospitals, phones).	Global Model
2. Local Training	Each node trains the model locally on its private dataset.	Nothing (Data stays put)
3. Update Upload	Nodes send only the updated parameters (gradients) to the server.	Local Updates
4. Aggregation	The server combines updates (e.g., Federated Averaging) to improve the model.	Improved Model

Federated Learning workflow: model moves, data stay local



Federated Learning (FL)

3. The Three Pillars of FL

- **Data Locality:** Data never leaves the original device. This eliminates the risk of data breaches during transit or central storage.
- **Decentralization:** Intelligence is built collectively, leveraging the diversity of worldwide data without needing to own it.
- **Network Efficiency:** Communicating model parameters is often much "lighter" than transferring gigabytes of raw data (medical images, sensor logs, etc.).

4. Why It Matters for Privacy

- Federated Learning ensures that **knowledge is shared while sensitive information remains private**. It represents the technological bridge between the need for high-performance AI and the fundamental right to data privacy

The Federated Averaging (FedAvg) Algorithm

A central server holds a global model with weights W . A large population of K clients, each with their own local dataset D_k .

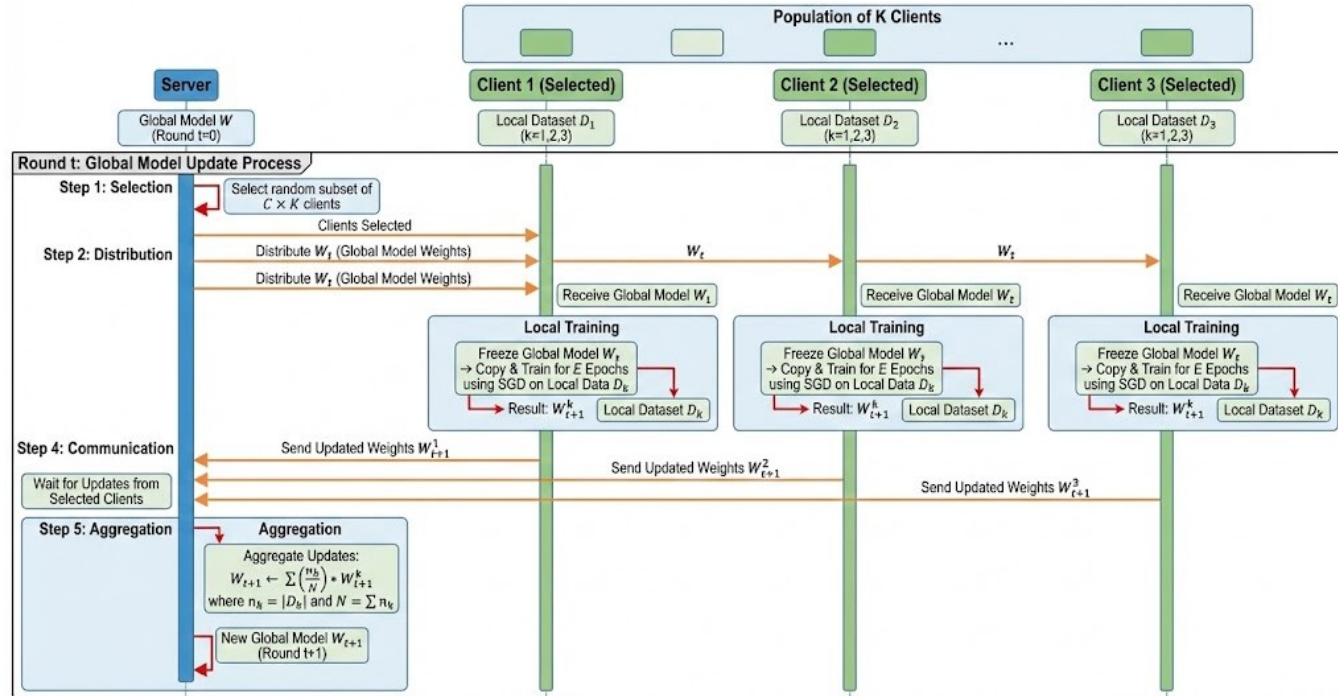
Round t :

- Selection: The server selects a random subset of $c \times K$ clients to participate.
- Distribution: The server sends the current global model weights, W_t , to all selected clients.
- Local Training: Each selected client k freezes the global model, copies it, and then trains it on its own local data D_k for E local epochs using an algorithm like Stochastic Gradient Descent (SGD). This results in a new set of local weights, W_{t+1}^k .
- Communication: Each client k sends its newly computed weights, W_{t+1}^k , back to the server.
- Aggregation: The server waits to receive updates from the clients. It then aggregates them to form the new global model for the next round, W_{t+1} , typically using a weighted average:

$$W_{t+1} \leftarrow \sum_{k=1}^{c \times K} \frac{n_k}{N} W_{t+1}^k$$

- where $n_k = |D_k|$ and $N = \sum_k n_k$.

The Federated Averaging (FedAvg) Algorithm



FL Topologies: Cross-Device vs. Cross-Silo

Our objective is now to categorize Federated Learning deployments based on the scale, reliability, and data characteristics of the participating nodes.

1. Cross-Device Federated Learning

Optimized for massive-scale consumer ecosystems.

Characteristics:

- **Scale:** Massive number of clients (reaching **millions** of devices).
- **Data Volume:** Very small dataset per client (user-specific interactions).
- **Reliability: Low.** Clients are mobile phones or IoT devices that frequently drop out due to poor connectivity, low battery, or user activity.
- **Orchestration:** Training only happens when devices are charging, on unmetered Wi-Fi, and idle.

Primary Example:

- **Mobile Keyboard Prediction:** Gboard learns new slang or personalized words across millions of devices without uploading private messages to the cloud.

FL Topologies: Cross-Device vs. Cross-Silo

2. Cross-Silo Federated Learning

Optimized for institutional and B2B collaboration.

Characteristics:

- **Scale:** Small number of clients (typically **2 to 100** organizations).
- **Data Volume:** Massive amounts of data per client (entire institutional databases).
- **Reliability: High.** Clients are robust servers or data centers with 24/7 availability and high-bandwidth interconnects.
- **Orchestration:** Training can be scheduled more aggressively; nodes are almost always available.

Primary Example:

- **Multi-Bank Fraud Detection:** Several banks collaborate to detect international money laundering patterns without sharing their private client transaction lists with competitors.
- **Medical Research:** Different hospitals collaborate to train a cancer-detection model on rare biopsy images while complying with patient confidentiality laws.

Summary Comparison

Feature	Cross-Device	Cross-Silo
Client Type	Mobile / IoT	Organizations / Servers
Client Count	Millions	10s – 100s
Data Distribution	Highly Fragmented	Consolidated at nodes
Focus	User Privacy	Business/Legal Privacy
Bottleneck	Wireless Link / Battery	Compute / Legal Agreements

Core Challenges & Advanced Federated Learning



Beyond the Idealized Algorithm

Let's transition from the theoretical elegance of Federated Learning to the "messy" reality of deployment. Next slides explore why FL is technically difficult to implement in production environments compared to simulation.

- **System Heterogeneity:** Managing a "fleet" of devices with different CPUs, RAM, and battery levels.
- **Statistical Heterogeneity:** Dealing with the Non-IID data problem (Client Drift).
- **Communication Bottlenecks:** Training over unstable, low-bandwidth wireless links.
- **Security & Poisoning:** Protecting the global model from malicious or "noisy" participants.

The "Non-IID" Problem: Statistical Heterogeneity

The most significant scientific challenge in Federated Learning is the Statistical Heterogeneity: how local data diversity breaks traditional optimization assumptions.

1. The IID Assumption (The Ideal Case)

- In traditional distributed learning (e.g., in a data center), we assume data is **Independent and Identically Distributed (IID)**.
- **The Logic:** Every node holds a representative "shuffled" slice of the total dataset.
- **The Math:** The local gradient at any node is an unbiased estimate of the global gradient.
- **Result:** Moving toward the local optimum is effectively the same as moving toward the global optimum.

The "Non-IID" Problem: Statistical Heterogeneity

2. The FL Reality: Profoundly Non-IID

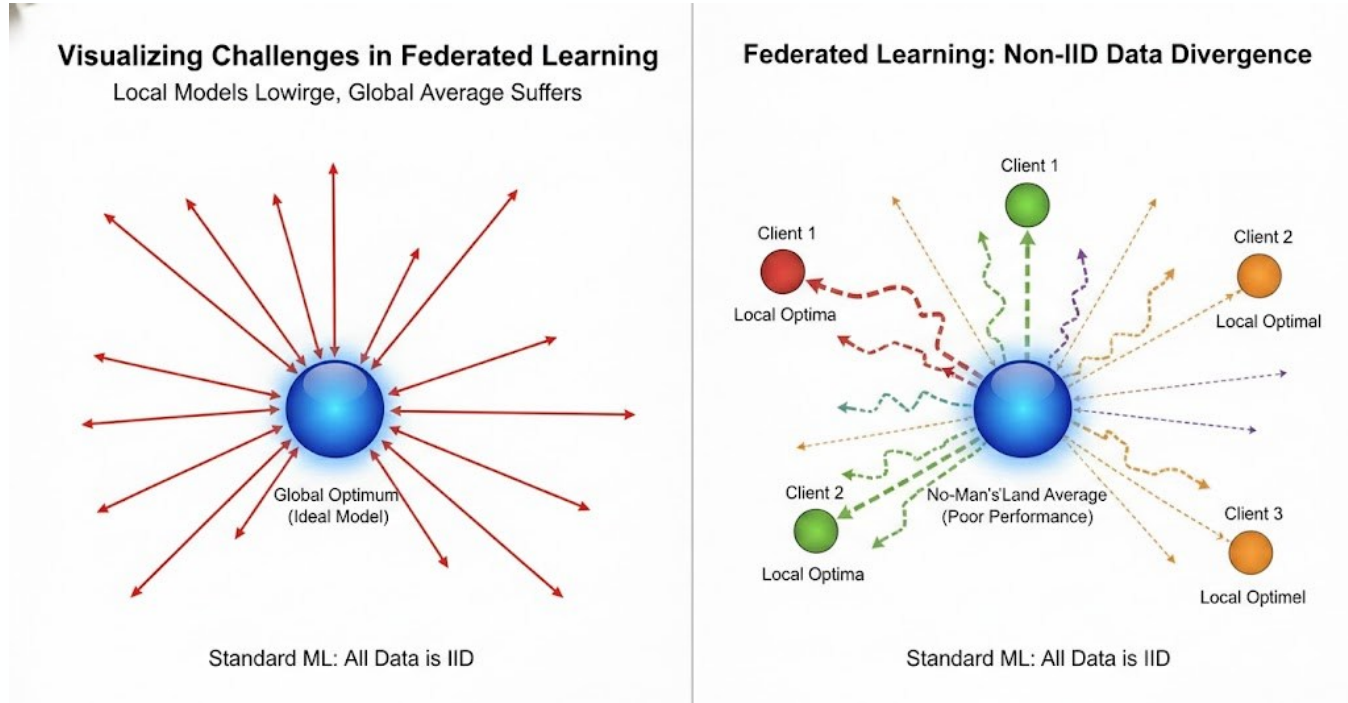
- In Federated Learning, data is generated locally based on specific user behavior or regional context.
- **Label Skew:** One hospital only sees rare cases; another sees common ones.
- **Feature Skew:** My photos are all of urban landscapes; yours are all of nature.
- **Quantity Skew:** One power-user generates 1,000 data points daily; another generates 5.
- **Temporal Skew:** User behavior changes over time (concept drift).

The "Non-IID" Problem: Statistical Heterogeneity

3. The Consequence: "Client Drift"

- When clients perform multiple local updates on skewed data before communicating back to the server:
 - **Local Divergence:** The local model optimizes for its specific "niche," moving toward a **local optimum** that is far from the **global optimum**.
 - **Weight Variance:** The updates (gradients) from different clients point in wildly different directions.
 - **The Penalty:** When the server averages these "drifted" models, the resulting global model may:
 - Slow down significantly in convergence.
 - Suffer from "catastrophic forgetting" of certain demographics.
 - Stall or diverge entirely, failing to reach a useful state.

The "Non-IID" Problem: Statistical Heterogeneity



The "Straggler" Problem: Systems Heterogeneity

Which are the systems-level challenges that occur when training models across a diverse fleet of edge devices?

The Reality: Clients are Not Equal

- In theoretical FL, we often assume every client completes their work at the same time. In reality, **systems heterogeneity** creates a fragmented training environment:
- **Hardware Variability:** Devices range from high-end flagship smartphones with dedicated AI accelerators to budget models with limited CPU cycles and minimal RAM.
- **Network Volatility:** Connectivity is rarely stable. Clients switch between Wi-Fi and 4G/5G, experience varying signal strengths, and may drop out mid-upload.
- **Opportunistic Availability:** Enterprise-grade FL (like Google's or Apple's) typically only runs when a device is:
 - Plugged into a power source (charging).
 - Connected to unmetered Wi-Fi.
 - Idle (user not interacting with the screen).

The "Straggler" Problem: Systems Heterogeneity

The "Straggler" Problem

- The **Straggler Problem** occurs when the central server initiates a synchronous training round.
- The server selects a subset of clients.
- The server must wait for the **slowest client** (the "straggler") to finish its local epochs and upload its gradient before the global model can be updated.
- **Impact:** A single client with a poor connection or an old processor can bottleneck the entire global network, extending training rounds from seconds to minutes or even hours.

Systems heterogeneity transforms FL from a purely mathematical optimization problem into a **complex distributed systems orchestration** challenge.

The Communication Bottleneck

Let's focus now on the challenge of communication efficiency in Federated Learning environments.

The Core Challenge

While Federated Learning avoids moving raw data, the "overhead" of the model itself presents a massive logistical hurdle:

- **Model Scale:** Modern deep learning architectures often contain millions (or billions) of parameters. Even if data stays local, the resulting model updates are geographically large.
- **Iteration Volume:** Training typically requires thousands of communication rounds between the server and clients.
- **Bandwidth Cost:** The cumulative cost of these transmissions often becomes the primary bottleneck, especially for edge devices with limited or unstable connectivity.

The Communication Bottleneck

Key Techniques for Improvement

1. Model Quantization

- **What it is:** Reducing the numerical precision of the model weights.
- **How it works:** Instead of using 32-bit floating-point numbers, weights are compressed into 8-bit integers (or even lower).
- **Benefit:** Dramatically reduces the size of each update packet without significantly sacrificing model accuracy.

2. Model Sparsification & Pruning

- **What it is:** Identifying and sending only the most critical information.
- **How it works:**
 - **Sparsification:** Sending only the most significant gradient updates (those above a certain threshold) rather than the whole model.
 - **Pruning:** Removing redundant or low-impact weights/connections from the model architecture entirely.
- **Benefit:** Minimizes the total number of bits transmitted per communication round.

Advanced Algorithm 1 — FedProx

FedProx introduce an advanced optimization framework designed specifically to tackle systems and statistical heterogeneity in Federated Learning.

1. The Problem: The "Straggler" & "Skew" Challenge

Standard FedAvg assumes all clients perform the same amount of work. In reality:

- **Systems Heterogeneity:** Devices have varying hardware, battery levels, and network speeds (the "Straggler" problem).
- **Statistical Heterogeneity:** Data is non-IID (Independent and Identically Distributed), leading to divergent local updates.

Advanced Algorithm 1 — FedProx

2. The Core Idea: The Proximal Term

FedProx modifies the local objective function by adding a **proximal term**. This term penalizes large deviations from the current global model.

Modified Local Objective:

$$\min_w h_k(w; w^t) = \min_w [F_k(w) + \frac{\mu}{2} |w - w^t|^2]$$

- $F_k(w)$: The standard local empirical risk (loss function) based on the client's private data.
- w^t : The state of the global model received from the server at the beginning of iteration t , serving as the reference anchor.
- $\frac{\mu}{2} |w - w^t|^2$: The **Proximal Term** (the "Leash"). The hyperparameter μ dictates the strength of this constraint; a higher μ forces the local model to stay closer to the global consensus, while a lower μ allows for more aggressive local adaptation. This term ensures that even if a client has highly skewed data, its local optimization process is mathematically "reminded" of the global objective.

Advanced Algorithm 1 — FedProx

3. The Intuition: A Mathematical Leash

- **The "Spring" Effect:** Think of the proximal term as a spring or leash. It allows local models to move toward their local optima but pulls back if they drift too far from the global consensus.
- **Tolerance for Partial Work:** Because the "leash" prevents divergence, the server can safely aggregate updates from clients that didn't finish all local epochs, rather than simply dropping them.

4. Key Benefits

- **Robustness:** More stable convergence in highly heterogeneous environments.
- **Flexibility:** Allows for variable amounts of local work across different devices without destabilizing the global model.

Advanced Algorithm 2 — SCAFFOLD

SCAFFOLD is an advanced algorithm designed to achieve stable and fast convergence by directly mitigating the phenomenon of "client drift."

1. The Challenge: Understanding "Client Drift"

- In Federated Learning with Non-IID data, local updates tend to pull the model toward the local optima of each client. Because these local optima are far apart, the average of these updates (the global model) often moves slowly or oscillates, a phenomenon known as **Client Drift**.

2. The Core Idea: Control Variates

SCAFFOLD (**S**tochastic **C**ontrolled **A**veraging **F**or **F**ederated **L**earning) introduces **control variates** to correct for the difference between the local gradient and the global gradient.

Key Components:

- **Local Control Variate (c_i)**: An estimate of the drift direction for client i . It tracks how much the local update "strays" from the global average.
- **Global Control Variate (c)**: The average of all local control variates, representing the general direction of the global model's trajectory.

Advanced Algorithm 2 – SCAFFOLD

3. The Intuition: The "Correction Factor"

- Imagine each client is trying to reach a destination but has a compass that is slightly off-center.
 - The Learning Process:** Each client doesn't just learn from its data; it applies a **correction factor** ($c - c_i$) to its local gradients.
 - Dynamic Adjustment:** If a client knows its data is skewed in a specific direction compared to the global model, it uses the control variate to "steer" its updates back toward the global consensus during local training.
 - Synchronization:** Both the model weights (w) and the control variates (c) are synchronized during communication rounds.

4. Mathematical Impact

- SCAFFOLD modifies the local update rule as follows:
$$g_i(w) \leftarrow \nabla F_i(w) + (c - c_i)$$
- This ensures that the expected local update remains an unbiased estimator of the global gradient, even when data is highly Non-IID.

5. Key Benefits

- Proven Convergence:** Offers provably better convergence rates compared to FedAvg, especially on highly heterogeneous (Non-IID) datasets.
- Communication Efficiency:** Often requires fewer communication rounds to reach a target accuracy because the updates are more "purposeful" and less prone to oscillation.

FedProx vs SCAFFOLD

Choose FedProx if:

- You have a **massive number of clients** and cannot afford to store state (control variates) for each one.
- Your network has many **stragglers** (devices with varying battery/CPU) that need to return partial results.
- You want a simple "drop-in" improvement over FedAvg with no extra communication overhead.

Choose SCAFFOLD if:

- You have a **fixed/smaller set of clients** (e.g., cross-silo FL between hospitals or banks).
- Reducing the **number of communication rounds** is your top priority (even if each round is twice as large).
- Your data is **extremely non-IID**, and FedProx/FedAvg are failing to reach high accuracy.

Application Deep Dive 1: FL for RAN Resource Management

Proactively allocate radio resources—such as those required for specific network slices—by leveraging high-precision predictions of traffic demand across the Radio Access Network (RAN).

The FL Approach (Network-Centric)

- **Clients:** Distributed 5G base stations (gNodeBs).
- **Local Data:** Each gNodeB maintains real-time traffic data, user connection patterns, and throughput metrics for its specific cell locally.
- **Training:** The gNodeBs collaboratively train a time-series forecasting model. Instead of streaming massive telemetry logs to a central cloud, only the localized model updates are shared.

The Strategic Benefit: Network-Wide Coordination

- **"Collective Intelligence, Local Privacy"** While each base station only "sees" its own coverage area, the federated approach allows the system to understand the broader network state:
- **Inter-Cell Coordination:** The global model identifies spatio-temporal traffic patterns (e.g., users moving from a residential cell to an office-district cell), allowing for better resource handovers.
- **Backhaul Efficiency:** Eliminates the need to transmit raw, high-frequency traffic data to a central controller, saving significant backhaul bandwidth.
- **Data Sovereignty:** Sensitive subscriber traffic patterns and location data remain strictly within the gNodeB, meeting stringent data protection requirements for mobile operators.

Application Deep Dive 2: FL for Intrusion Detection

Detect new and emerging cyberattacks across a network of independent organizations (e.g., a consortium of banks or healthcare providers) without exposing internal infrastructure details.

The FL Approach (Cross-Silo)

- **Clients:** Individual enterprise networks (the "silos").
- **Local Data:** Each organization maintains its own private network traffic logs, connection metadata, and system alerts locally.
- **Training:** Organizations collaboratively train a shared anomaly detection model. Only model weight updates are exchanged with a central server—never the raw logs.



The Strategic Benefit: Collaborative Immunity

- **"Immunization via Exposure"** If one organization is hit by a novel zero-day attack, the local model learns the new signature/anomaly. Once these updates are aggregated into the global model:
- **Proactive Protection:** The entire network is updated to recognize the threat.
- **Zero Data Exposure:** All other participating organizations gain protection against the attack without any sensitive internal traffic logs ever leaving their respective firewalls.
- **Scale:** Small organizations benefit from the sophisticated threat detection capabilities of larger ones, and vice versa.

Transfer Learning and Split Learning



Transfer Learning (TL): Learning from Related Domains

Transfer Learning reuses knowledge learned in one context to improve learning in another related context.

Source domain / source task:

The environment where knowledge is first learned.

Target domain / target task:

The new environment where the model must operate.

Why it matters in networks:

Wireless environments change continuously due to mobility, interference, topology changes, and variable traffic demand. Training a new model from scratch for every new cell, slice, or service is often too slow and too data-hungry.

Typical network example:

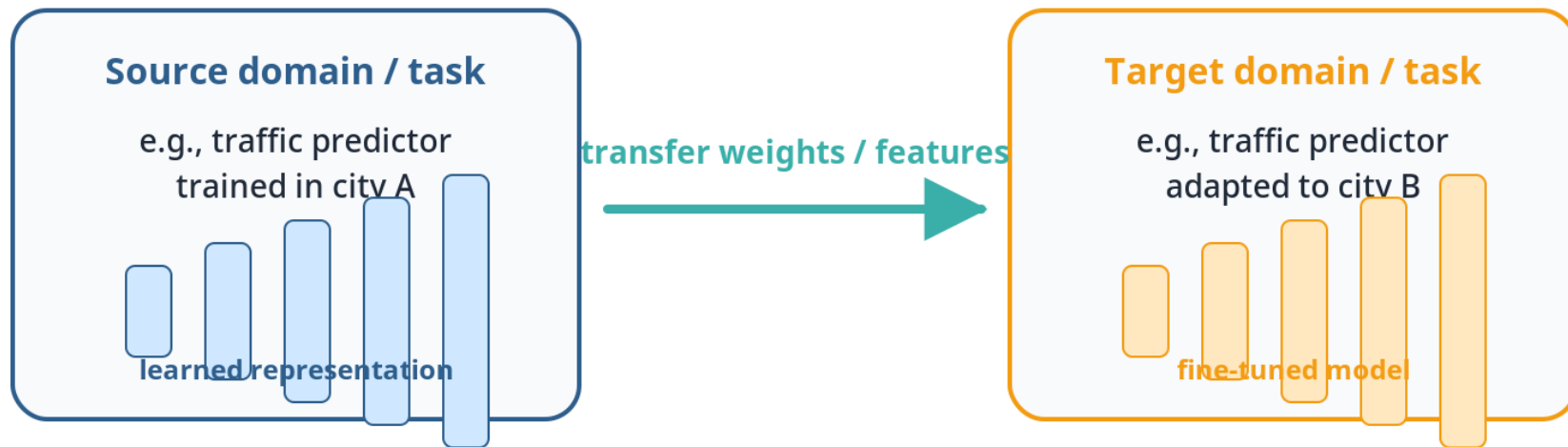
A traffic prediction model trained in a dense urban area can be adapted to a new district with fewer labeled samples.

Key message:

TL is useful when the new problem is different, but not completely unrelated, to a previous one.

The TL reference defines TL as learning a target predictive function by leveraging knowledge from a source domain/task when either the domain or the task differs.

Transfer Learning: reuse knowledge from a source task/domain



Target model starts closer to a good solution: less data, less time, less computation

Why Transfer Learning is Useful for Wireless Networks

Transfer Learning addresses four recurring limitations of conventional ML in networks:

1. Lack of labeled data

New cells, new services, and new devices often have little historical data.

2. Fast adaptation

The network may need to adapt before enough new samples are collected.



3. Reduced computation

Instead of training from scratch, the target model starts from a useful representation.

4. Lower communication overhead

Transferring model knowledge is usually lighter than collecting and transferring raw datasets.

Examples in network applications:

- Traffic load prediction
- Channel quality prediction
- Signal classification
- Spectrum sensing
- Caching and content popularity prediction
- Security anomaly detection

Transfer Learning Taxonomy

Transfer Learning can be classified according to what changes between source and target.

Type	What changes?	Network example
Inductive TL	Same/similar domain, different task	From traffic prediction to congestion classification
Transductive TL	Same task, different domain	Same prediction model adapted from one city to another
Unsupervised TL	No labels in source/target	Clustering user mobility patterns across cells

Other view: what is transferred?

- **Instance-based TL:** reweight source samples.
- **Feature-based TL:** learn common feature representation.
- **Parameter-based TL:** reuse model weights or hyperparameters.
- **Relational TL:** transfer structural relationships.

Key message: The central design questions are: what to transfer, when to transfer, and how to transfer.

Deep Transfer Learning

In deep learning, the transferred knowledge is usually embedded in the network architecture and weights.

Common strategies:

- **1. Pre-trained model as initialization**
Start from a model trained on a large source dataset and fine-tune it on the target network data.
- **2. Feature extractor**
Reuse the first layers of a neural network to extract general features, then train a new classifier/regressor.
- **3. Fine-tuning**
Freeze some layers and retrain others depending on how similar the source and target domains are.

Network intuition:

Lower layers learn general patterns, while higher layers learn task-specific representations.

Example:

A model trained to classify modulation schemes in one RF environment can be fine-tuned for another propagation scenario.

Transfer Learning Risks: Negative Transfer

Transfer Learning is not always beneficial.

Negative Transfer

If the source and target domains are too different, transferred knowledge can degrade performance instead of improving it.

Examples:

- A channel model trained in sub-6 GHz may not transfer well to mmWave without adaptation.
- A traffic predictor trained for business districts may fail in residential areas.
- A security classifier trained on one attack profile may miss new attack behaviors.

Mitigation strategies:

- Measure source-target similarity before transfer.
- Use small target-domain validation sets.
- Fine-tune only selected layers or parameters.
- Monitor for concept drift after deployment.

Beyond Horizontal FL: Vertical FL and Split Learning

The current FL discussion mostly assumes **horizontal FL**:

- Different clients hold different samples.
- All clients share the same feature space.
- Example: many phones training the same keyboard model.

But in many real systems, data is not organized this way.

Vertical FL

Different parties hold different features about the same entities.



Example: one hospital has images, another has lab tests, another has clinical history.

Split Learning

Different parties execute different parts of the neural network.

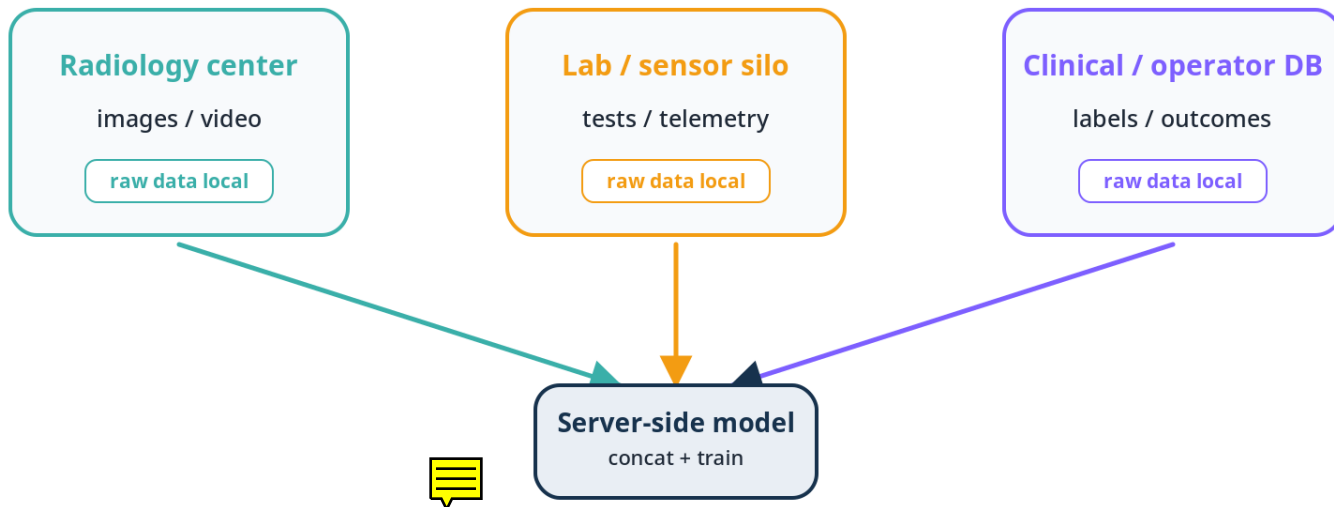
The client keeps raw data and early layers; the server runs the remaining layers.

Key message:

When a single party does not have all the data or resources to train alone, FL must be extended with vertical or split approaches.

From Vertical FL to Split Learning

Example: patient/device/user ID aligns heterogeneous data sources



Useful when no single party has all features or enough resources

Split Learning (SL): Split the Neural Network

Split Learning partitions a neural network across client and server.

Client side:

- Stores raw data.
- Executes the first layers of the neural network.
- Sends only intermediate activations.

Server side:

- Receives activations from the split layer.
- Executes the remaining layers.
- Computes the loss and backpropagates gradients.

What is exchanged?

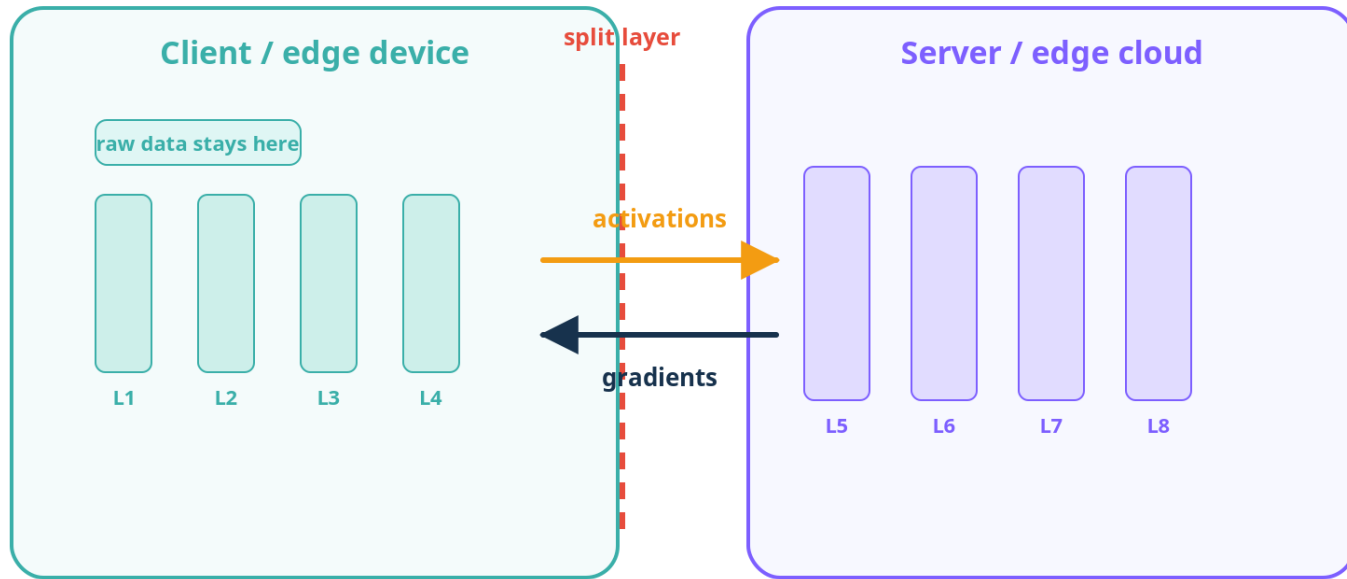
- Not raw data.
- Not full model weights.
- Only activations and gradients at the split layer.



Key message:

- FL moves the model to the data.
- SL moves only part of the model to the data.

Split learning: client side layers + server side layers



No raw data transfer; only split-layer signals cross the network

Split Learning Workflow

Step 1 — Forward pass at the client

The client processes local data through the first layers of the neural network.

Step 2 — Activation sharing

The client sends the output of the split layer to the server.

Step 3 — Forward pass at the server

The server completes the prediction using the remaining layers.

Step 4 — Loss computation

The server computes the loss, depending on where labels are stored.

Step 5 — Backward pass

The server sends gradients back to the client at the split layer.

Step 6 — Local update

The client updates its own layers; the server updates its layers.

Key message:

Training is collaborative, but raw input data never leaves the client.

Split Learning: Benefits and Limitations

Benefits

- Reduces client computation: the client trains only the first layers.
- Useful for resource-constrained IoT and edge devices.
- Keeps raw data local.
- Can support vertically partitioned or multi-modal data.
- Can be combined with privacy mechanisms for activation sharing.

Limitations

- Requires frequent client-server interaction.
- Activations may still leak information if not protected.
- Latency can be critical because forward and backward passes cross the network.
- Server becomes part of the training path, not only an aggregator.

Key message:

SL is attractive when clients are weak, but the network link must be reliable and low-latency.

FL vs TL vs SL: When to Use What?

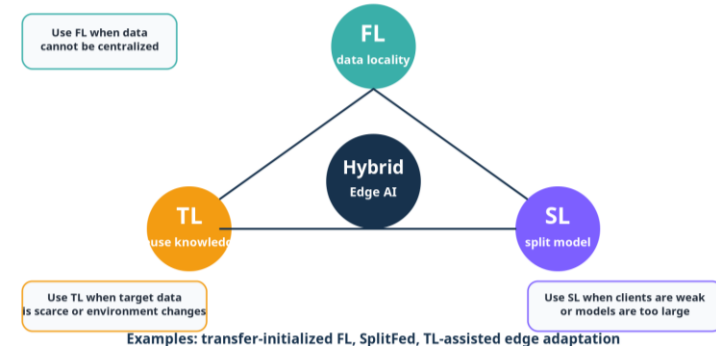
Paradigm	Best when...	What moves?	Main bottleneck
Federated Learning	Many clients have local data and can train the same model	Model updates	Communication rounds, non-IID data
Transfer Learning	A related model/task already exists	Knowledge, weights, features	Negative transfer, domain mismatch
Split Learning	Clients cannot train full models locally	Activations and gradients	Latency, activation leakage
Centralized ML	Data can legally and efficiently be centralized	Raw data	Privacy, bandwidth, cloud latency

Choosing the right distributed learning paradigm

Decision rule:

- Use **FL** when privacy prevents data sharing but clients can train locally.
- Use **TL** when data is scarce or the environment changes.
- Use **SL** when clients are resource-constrained or the model is too large.
- Combine them when needed: e.g., transfer-initialized FL, or SplitFed.

Key message: Modern network intelligence is hybrid: the best architecture often combines FL, TL, SL, MEC, and privacy-enhancing technologies.



Summary of DML and Final Bridge

Let's wrap up the Distributed Machine Learning (DML) topic and synthesize the learning objectives of the entire module.

Paradigm Shift: DML, and specifically Federated Learning, is a critical paradigm for building intelligent systems on decentralized, private data.

Key Constraints: While powerful, it requires addressing significant challenges in:

- **Heterogeneity:** Diverse device capabilities and data distributions.
- **Communication:** High overhead in model parameter exchange.
- **Privacy:** Ensuring data remains local while model intelligence is shared.

The Big Picture (Module 3 Synthesis)

"In this module, we have explored the core paradigms that define modern network applications:"

- **MEC:** For low-latency distributed compute.
- **Slicing:** For guaranteed connectivity and resource isolation.
- **Multimedia:** Advanced protocols for high-bandwidth, demanding services.
- **DML:** The distributed intelligence required to manage and optimize it all.

Bridge to Module 4: Automation

"Now that we have all these powerful, independent paradigms, how do we integrate and manage them holistically? How do we build a network that can truly manage itself?"

Looking Ahead: This is the central theme of our final module on **AI-driven Automation** and the future of autonomous networking.



UNIVERSITÀ
DEGLI STUDI
FIRENZE

Da un secolo, oltre.



HR EXCELLENCE IN RESEARCH

03.4: Distributed Machine Learning

Network Applications

LM in Ingegneria Informatica

Daniele Tarchi

Dipartimento Ingegneria dell'Informazione

daniele.tarchi@unifi.it